UNIT-II

Intel 8085

Contents

2.1. Functional Block Diagram, Pin Configuration, Description of each Block: Registers, Flag (Description of each Flag), Data and Address Bus including Bidirectional Address/Data Bus, Timing and Control Unit, Interrupts (Introduction Only), Instructions: Op-Code and Operands, Addressing Modes, Instructions and Data Flow

2.2. 8085 Instructions: Data Transfer Instructions, Arithmetic and Logic Instructions, Branching and Machine Instructions

2.3. Basic Assembly Language Programming using 8085 Instruction Sets Addition, Subtraction, Multiplication and Division, Simple Sequence Programs, Array Searching and Sorting using Branching and Looping

Features of 8085 Microprocessor

- The Intel 8085A is a complete 8 bit parallel processor.
- Accumulator based Microprocessor
- 40 pins 8085A, requires +5V single power supply
- Operate with 3MHz, single phase clock
- It includes array of registers, the arithmetic logic unit, instruction register and decoder, timing and control circuits, Interrupt control and Serial I/O control.
- They are linked by an internal data bus

Internal Architecture of 8085 Microprocessor

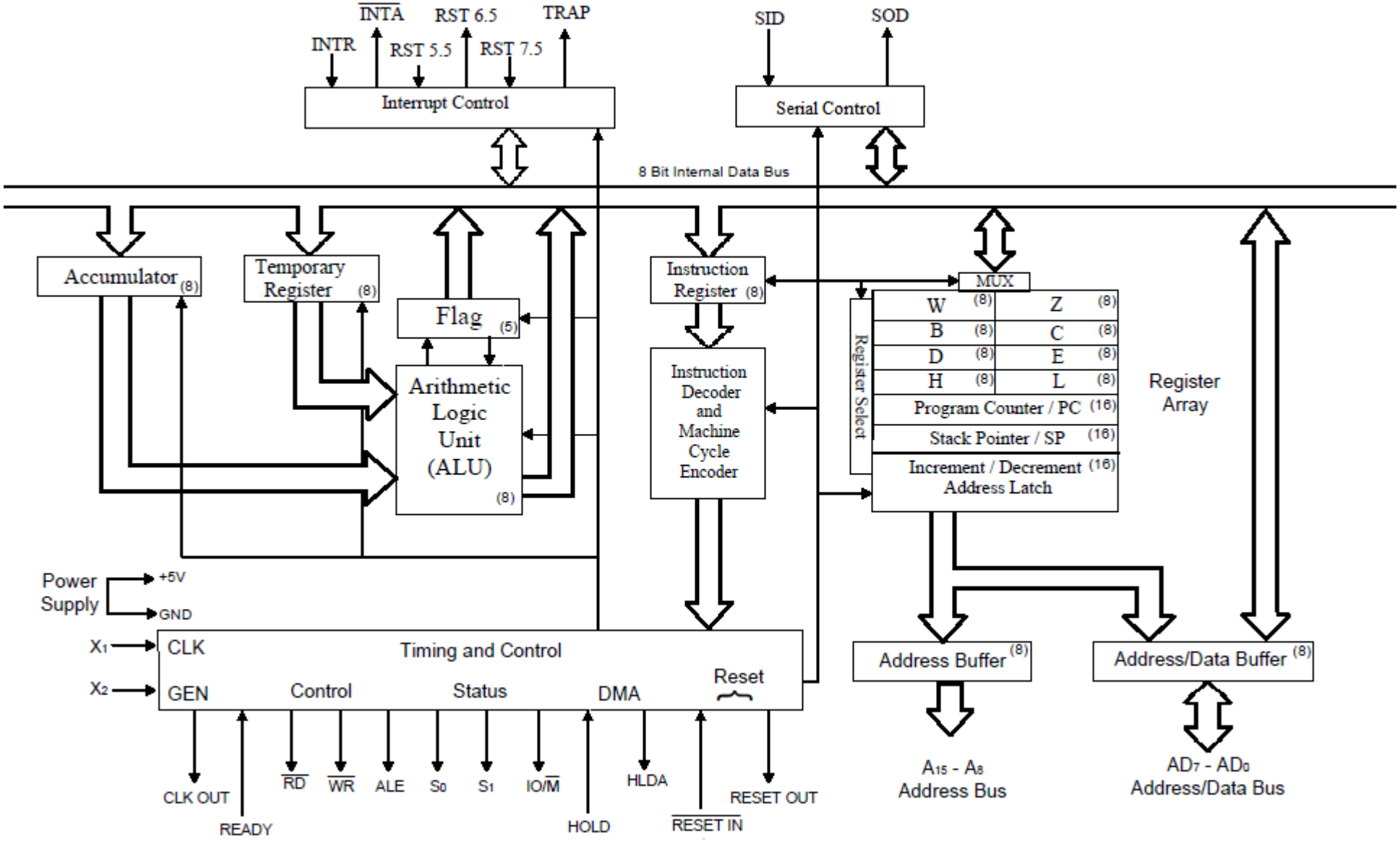
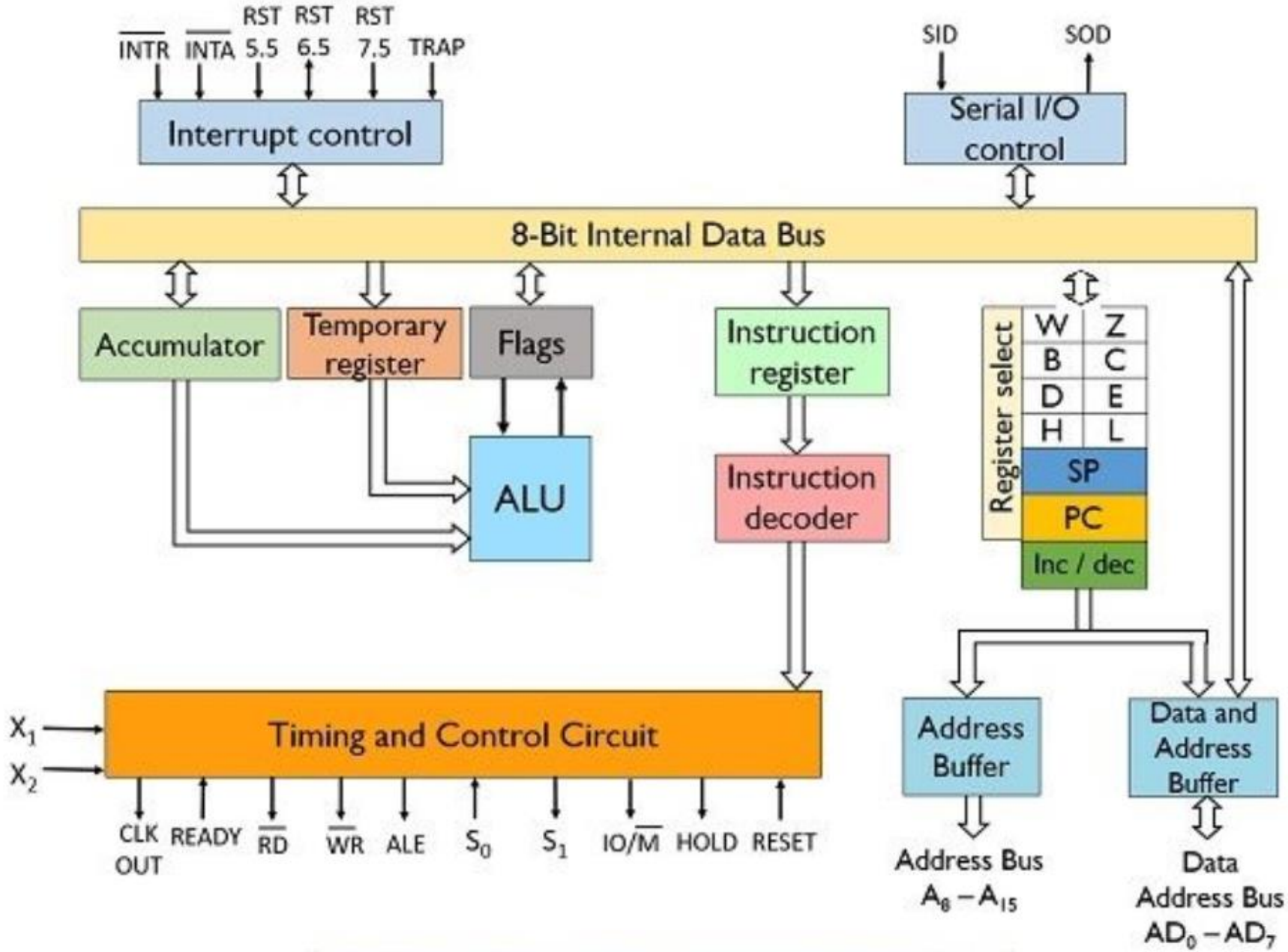


Fig: Internal architecture of 8085A microprocessor (Functional Block Diagram)



Architecture of 8085 Microprocessor

Internal Architecture of 8085 Microprocessor

1. Register Arrays:
2. Arithmetic Logic Unit
3. Instruction Register & Decoder
4. Timing & Control Unit
5. Interrupt Control
6. Serial I/O Control
7. Bus (Address and Data)

1. Register Array

- Accumulator
  - 8 bit register
  - To perform logic and arithmetic operation and also store 8 bit data.
  - Result of arithmetic operation is stored on accumulator
- Temporary Register
  - 8 bit register
  - Not accessible to programmer
  - Used to hold data during arithmetic/logic operation
  - W and Z are the registers used to hold the results of the operation

1. Register Array

- General Purpose Registers
  - Six general purpose registers, each can store 8 bit data
  - B, C, D, E, H and L (each of 8 bit)
  - Also can be grouped in pairs as register pair B (includes B & C), D (includes D & E), and H (includes H & L) to perform 16 bit operation
  - Accessible to programmer
  - H and L can be utilized in indirect addressing mode where the memory location is specified by the content of H and L
- Stack Pointer (SP)
  - Stack is the area of memory set aside to store data temporarily
  - Stack pointer contains the address of the beginning of the stack which is 16 bit address

1. Register Array

- Program Counter (PC)
  - 16 bit register
  - Points to the memory location from where the next byte of instruction is to be fetched
  - Depending upon the instruction size PC is increased by 1, 2, 3 to point the memory location of next instruction
- Flags
  - Five flip flops are set or reset according to the result of arithmetic or logical operations
  - CY (Carry), P (Parity), AC (Auxiliary Carry), Z (Zero) and S (Sign) flags holds the status of different states

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
S	Z	X	AC	X	P	X	CY

Flags

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
S	Z	X	AC	X	P	X	CY

- **Carry**: - If the last operation generates a carry its status will 1 otherwise 0. It can handle the carry or borrow from one word to another.
- **Parity:** - If the result of the last operation has even number of 1's (even parity), its status will be 1 otherwise 0.
- **Auxiliary carry: -** If the last operation generates a carry from the lower half word (lower nibble), its status will be 1 otherwise 0. Used for performing BCD arithmetic.
- **Zero:** - If the result of last operation is zero, its status will be 1 otherwise o. It is often used in loop control and in searching for particular data value.
- **Sign: -** If the most significant bit (MSB) of the result of the last operation is 1 (negative), then its status will be 1 otherwise 0.

2. Arithmetic Logic Unit (ALU)

- Performs mathematical computing functions like addition, subtraction, logical functions like AND, OR, Shifting etc.
- Temporary registers are used to hold the data during those operation and result is stored in accumulator
- Flags are set or reset according to the result of the operation

3. Instruction Register & Decoder

- Instruction register receives the operation codes (Opcodes) of instruction and passes to the instruction decoder and machine cycle encoder
- Instruction decoder decodes the opcode so that the microprocessor knows which type of the task to be performed during execution
- These registers are not accessible to programmer

4. Timing & Control Unit

- Synchronizes all the microprocessor operations with the clock
- Generates the control signals necessary for communication between microprocessor and peripherals
- Signals:
  - Address Latch enable (ALE)
  - Clock (CLK, GEN, CLK OUT, READY)
  - Control ($\overline{RD}$, $\overline{WR}$)
  - Status (S_1, S_0, IO/$\bar{M}$)
  - DMA (HOLD, HLDA)
  - Reset ($\overline{RESET\ IN}$, RESET OUT)

5. Interrupt Control

- Used to interface external devices or peripherals with the processor for data communication
- Interrupt Signals are INTR, $\overline{INTA}$, RST 7.5, RST 6.5, RST 5.5, TRAP
- INTR-Interrupt Request, INTA-Interrupt Acknowledge

6. Serial I/O Control

- Performs parallel to serial and serial to parallel conversion for serial data communication with serial devices
- Serial I/O Control signals:
  - SID (Serial data input)
  - SOD (Serial data output)

7. Data Bus and Address Bus

- The 8085 microprocessor utilizes a 16-bit unidirectional address bus (A_8-A_{15} and multiplexed AD_0 - AD_7) to address 64KB of memory and an 8-bit bidirectional data bus (AD_0 - AD_7) to transfer data.

- To reduce pin count, the lower-order address bus (A_0 - A_7) is multiplexed with the data bus (D_0 - D_7) on pins (AD_0 - AD_7) , requiring an external latch to demultiplex them.

7. Data Bus and Address Bus

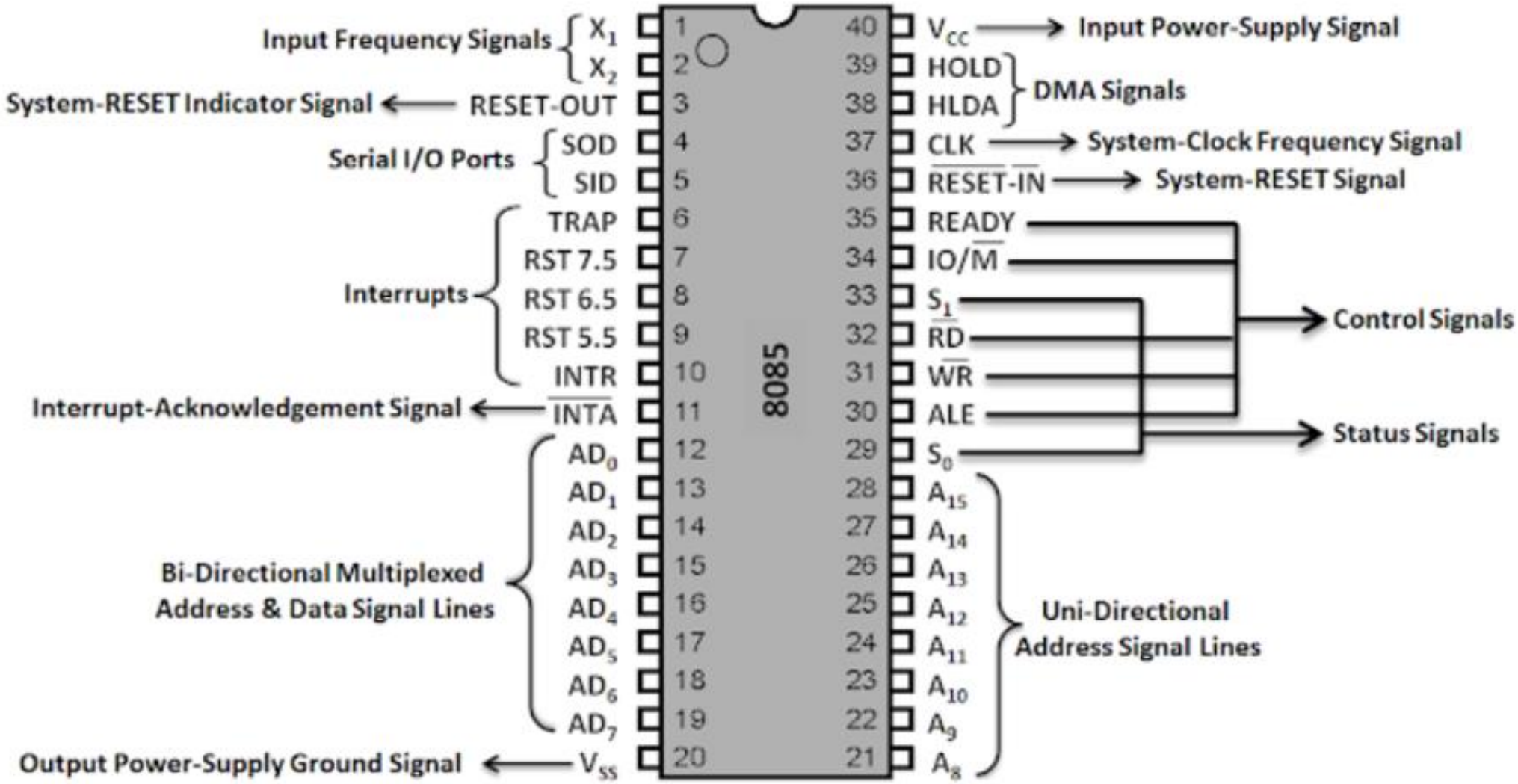
- Address Bus (16-bit)
  - Function: Carries memory addresses from the CPU to memory or I/O devices.
  - Width: 16-bit (A_0 - A_{15}), providing a 64KB (2^16) addressing space.
  - Direction: Unidirectional (CPU ->Peripheral/Memory).
  - Structure:Higher Order: (A_8 - A_{15}) (dedicated lines).
  - Lower Order: (A_0 - A_7) (multiplexed with data bus).
- Data Bus (8-bit)
  - Function: Transfers data between the CPU, memory, and I/O devices.
  - Width: 8-bit (D_0 - D_7).
  - Direction: Bidirectional (CPU <-> Peripheral/Memory).
  - Operation: Uses 8 parallel lines to transfer 1 byte at a time.

Multiplexing and DemultiplexingMultiplexed Bus:

- To save pins, the 8085 uses AD_0 - AD_7 lines to act as both the lower-order address bus and the 8-bit data bus.
- Demultiplexing: An external latch (e.g., 74LS373) is used, controlled by the ALE (Address Latch Enable) signal, to separate the lower address byte from the data byte.
- The address is held stable while the data bus is used for transfer.

Bus Type	Width	Type	Usage
Address	16-bit	Unidirectional	A_8-A_{15} (High), AD_0-AD_7 (Low-multiplexed)
Data	8-bit	Bidirectional	AD_0-AD_7 (Multiplexed with Address)

Pin Configuration of 8085 Processor



Grouping of Pins in 8085

1) Input Frequency Signals.
2) System RESET Indicator Signal.
3) Serial I/O Ports.
4) Interrupts.
5) Interrupt-Acknowledgement Signal
6) Bi-Directional Multiplexed Address & Data Lines.
7) Output Power-Ground Signal.
8) Uni-Directional Address Lines.
9) Status Signals.
10) Control Signals.
11) Reset Signals.
12) System-Clock Frequency Signal.
13) DMA Signals.
14) Input Power-Supply Signal.

Instruction Format

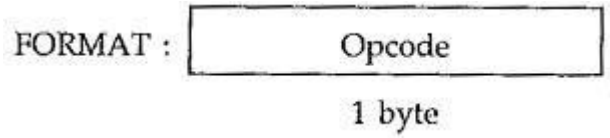
- Instruction is a command given to the processor which is basically a binary pattern design to perform a specific task
- Each instruction has two parts:
  - Operation Code (Opcode)
    - It specifies how the operations are to be manipulated.
    - Says how the task to be performed
  - Operand
    - These are the data to be operated upon or the address or location of the operands
    - The operands may be 8 bit data or 16 bit data or internal registers or 8 bit or 16 bit addresses.
    - In some cases the operand in the instruction is implicit. E.g. CMA

Instruction:	Opcode	Operands
Example:	MVI A,12	

Instruction Format

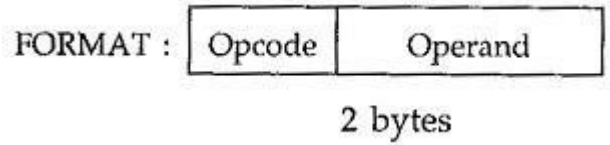
- Instruction word size:

1. One Byte Instruction:
   - Opcode and operand both are in same byte
   - E.g. MOV A,B



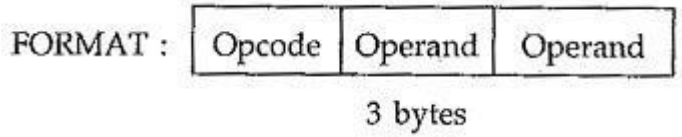
Instruction:	MOV A,B	ADD B
Opcode	78	80
Operand	-	-

2. Two Byte Instruction:
   - First byte is opcode while 2nd byte is operand
   - E.g. MVI A,20H



Instruction:	MVI A,20H	ADI 55H
Opcode	3E	C6
Operand	20	55

3. Three Byte Instruction:
   - First byte is opcode and 2nd and 3rd bytes are operands (may be 16 bit data or address)
   - E.g. LDA 9010H



Instruction:	LXI B,1234H	LDA 9010H
Opcode	01	3A
Operand (Lower Byte)	34	10
Operand (Higher Byte)	12	90

Opcode Format

- The opcode is unique for each Instruction
- Contains the information about operation, register to be used, memory to be used etc.
- The 8085A identifies all operations, registers and flags with a specific code
- All operations, registers and status flags are identified with a specific code
- E.g. MOV B,C = 01 000 001= 41H
- ADD B = 10000 000= 80H
- LXI H, 1234H = 00 10 0001= 21H

Code	Register Pair	Code	Register
00	BC	000	B
01	DE	001	C
10	HL	010	D
11	AF or SP	011	E
		100	H
		101	L
		110	M (Memory)
		111	A

Sr. No.	Function	Operation code							
		B_7	B_6	B_5	B_4	B_3	B_2	B_1	B_0
1	MVI r, data	0	0	D	D	D	1	1	0
2	LXI rp, data	0	0	D	D	0	0	0	1
3	MOV rd, rs	0	1	D	D	D	S	S	S

D = Destination, S = Source

Data Format

- The operand is an another name for data. It may appear in different forms :
- **Addresses**
- **Numbers/Logical data and**
- **Characters**
- ASCII Code
  - 7 bit alphanumeric code that represents decimal nos., English alphabet and other non-printable characters
- BCD Code
  - Binary coded decimal has 10 digits, i.e. 4 bits to represent from 0000 to 1001 (1010 to 1111 are invalid
- Signed Integer
  - MSB is used for sign and remaining 7 bit represents data (00H to 7F are positive integer 80H to FFH are negative in this case
- Unsigned Integer
  - 8 bits to represent data (00H to FFH)

Addressing Modes of 8085 Microprocessor

- Various format of specified operands for an instruction is called addressing mode
- The operands can be specified by the address or data

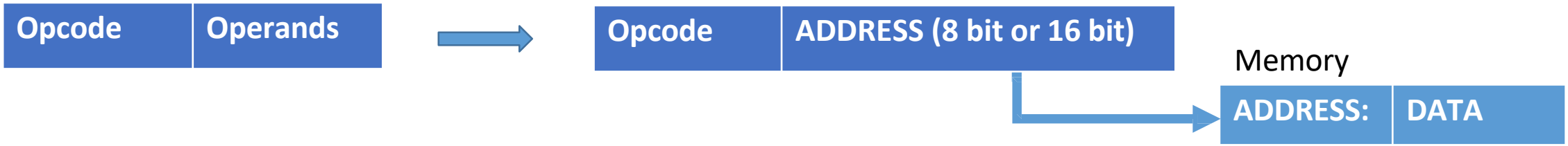
Opcode	Operands

1. Immediate Addressing Mode
2. Direct Addressing Mode
3. Register Direct Addressing Mode
4. Register Indirect Addressing Mode
5. Implied or Inherent Addressing Mode

1. Immediate Addressing Mode

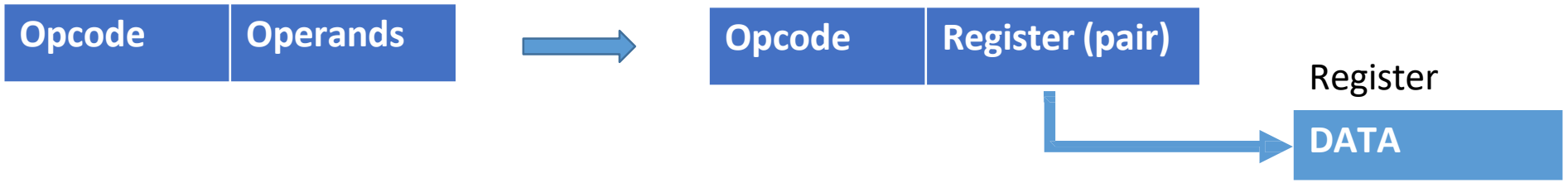
- It uses one byte or two byte of data that follows opcode (1 byte)
- E.g.
  - MVI B,02H (Move immediate data 02H to register B)
  - ADI 82H (Add immediate data 82H with accumulator)
  - LXI B,8050H (Load register pair B (BC) with immediate data 8050H)

2. Direct Addressing Mode



- It uses effective address of memory or I/O ports as a part of instruction
- Memory address (16 bit)
- I/O address (8 bit)
- E.g.
  - LDA 9000H (Load accumulator with data from memory address 9000H)
  - STA 8050H (Store data of accumulator to memory address 8050H)
  - IN 40H (read/load data from i/o port address 40H to accumulator)

3. Register Direct Addressing Mode



- It specifies the register or register pair that contains data
- E.g.
  - MOV B,C (Move data from register C to register B)
  - ADD B (add data of register B with accumulator)

4. Register Indirect Addressing Mode

- The operand part specifies the location whose content is the address of data
- i.e. the operand contains the address of data rather than data itself
- E.g.
  - LDAX B (Load data to accumulator from memory address (content of BC register pair)
  - MOV A,M (Move data to accumulator from memory address (content of HL register pair)

5. Implied or Inherent Addressing Mode

- Don’t have operands or simply assumed accumulator as an operand
- E.g.
  - CMA (complement Accumulator)
  - RLC (rotate accumulator left)
  - STC (Set carry flag 1)

8085 Instruction Set

1. Data Transfer Instruction
2. Arithmetic Operation
3. Logical operation
4. Branching Instruction
5. Miscellaneous Instruction

A. Data Transfer Instructions

- Group of instruction copy data from a source location to destination location without modifying the contents of the source.
- Transfer of data may be between the registers or between register and memory or between an I/O device and accumulator
- None of these instructions changes the flag

a) Data Transfer Group

1. MOV R_d, R_s	**2.** MOV R, M & MOV M, R	**3.** MVI R, data
4. MVI M, data	**5.** LXI R_P, 16 bit data	**6.** LDA addr
7. STA addr	**8.** LHLD addr	**9.** SHLD addr
10. LDAX R_P	**11.** STAX R_P	**12.** XCHG
13. IN 8 bit address	**14.** OUT 8 bit address	

A. Data Transfer Instructions

1. Immediate Data Transfer

a) MVI R, 8-bit DATA

- load 8 bit data to specified register R
- 2 byte instruction
- R may be Accumulator A or registers B,C,D,E,H,L
- E.g.
  - MVI A,30H ; A←30H
  - MVI B,FFH ; B←FFH

b) LXI R_p, 16-bit data

- Load 16 bit data to specified register pair R_p
- 3 byte instruction
- R_P may be B, D, H
- E.g.
  - LXI B,1122H ; C← 22H B← 11H
  - LXI H,5678H ; L← 78H H← 56H

```
Mnemonic- MVI A,32H
Opcode- MVI
Operand- A, 32H
Hex Code- 3E
          32
Binary code- 0011 1110
             0011 0010
```

A. Data Transfer Instructions

2. Register to Register data transfer
   a) MOV R_d,R_S
   - Move data from source register R_S to destination register R_d
   - 1 byte instruction
   - R_d and R_s may be Accumulator A or registers B,C,D,E,H,L
   - E.g.
     - MOV A,B ; A ← B
     - MOV D,H ; D ← H

A. Data Transfer Instructions

3. Data transfer to/from Memory
   a) MVI M,8-bit data
      - load memory immediate
      - 2 byte instruction
      - Loads the 8-bit data to the memory location whose address is specified by the contents of HL pair.
      - E.g.
        - MVI M,55H ;[HL] ← 55H {suppose HL= 8050H, [8050H]←55H}

A. Data Transfer Instructions

3. Data transfer to/from Memory

b) MOV M,R

- Move to memory from register
- Copy the contents of the specified register to memory. Here memory is the location specified by contents of the HL register pair.
- E.g.
  - MOV M,B ; [HL] ← B {suppose HL= 8060H and B= 11H then [8060H]←11H}

b) MOV R,M

- Move to register from memory
- Copy the contents of memory location specified by HL pair to specified register.
- E.g.
  - MOV A,M ; A ← [HL] {suppose HL= 8060H A ← [8060H]}

A. Data Transfer Instructions

3. Data transfer to/from Memory

d) LDA 16-bit ADDRESS

- Load Accumulator direct
- 3-byte instruction
- Loads the accumulator with the contents of memory location whose address is specified by 16 bit address.
- E.g. LDA 9000H ; A ← [9000H]

d) STA 16-bit ADDRESS

- Store accumulator content direct
- 3-byte instruction
- Stores the contents of accumulator to specified address
- E.g. STA 9050H ;[9050H] ← A

A. Data Transfer Instructions

3. Data transfer to/from Memory

f) LDAX R_p

- Load accumulator indirect
- 1 byte instruction
- Loads the contents of memory location pointed by the contents of register pair to accumulator
- E.g. LDAX B ; A ← [BC] {suppose B=80, C=50, A← [8050]=8-bit data

g) STAX R_p

- Store accumulator content indirect
- 1-byte instruction
- Stores the content of accumulator to memory location pointed by the contents of register pair
- E.g. STAX D ; [DE] ← A {suppose D=90, E=50, [9050] ←A}

A. Data Transfer Instructions

3. Data transfer to/from Memory

h) LHLD 16-bit ADDRESS

- Load HL directly
- 3-byte instruction.
- Loads the contents of specified memory location to L –register and contents of next higher location to H-register.
- E.g. LHLD 8050H ; L ← [8050] , H ← [8051]

h)SHLD 16 bit ADDRESS

- store HL directly
- Opposite to LHLD
- Stores the contents of L-register to specified memory location and contents of H register to next higher memory location
- E.g. SHLD 8060 ; [8060] ← L, [8061] ←H

A. Data Transfer Instructions

4. Data Exchange between register

   j) **XCHG**

   - Exchange
   - Exchanges DE pair with HL pair.
   - E.g. XCHG ; HL ←→ DE { L←→E and H ←→ D}

A. Data Transfer Instructions

5. Data transfer to/from I/O Ports

a) IN 8-bit ADDRESS

- 2-byte instruction
- Read data from the input port address specified in the second byte and loads data into the accumulator
- input port content to accumulator
- E. g. IN 40H ; A ← [40H]

a) OUT 8-bit ADDRESS

- 2-byte instruction
- Copies the contents of the accumulator to the output port address specified in the 2nd byte
- accumulator to output port
- E. g. OUT 40H ; [40H] ← A

B. Arithmetic Operation

The arithmetic operation add and subtract are performed in relation to the contents of accumulator. The features of these instructions are:

- They assume implicitly that the accumulator is one of the operands
- They modify all the flags according to the data conditions of the result.
- They place the result in the accumulator.
- They do not affect the contents of operand register or memory.
  - ➢ But the INR and DCR operations can be performed in any register or memory. These instructions
    - Affect the contents of specified register or memory.
    - Affect the flag except carry flag

B. Arithmetic Operation

- performs various arithmetic operations such as addition, subtraction, increment and decrement
- Affects the content of flags

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
S	Z	X	AC	X	P	X	CY

- **Carry**: - If arithmetic operation results carry CY=1 (set), 0 (Reset) otherwise
- **Parity:** - If the result has even number of 1's (even parity), then P= 1 otherwise 0.
- **Auxiliary carry: -** if carry from the lower nibble, AC= 1 otherwise 0. Used for performing BCD arithmetic.
- **Zero:** - If the result of last operation is zero, Z= 1 otherwise 0.
- **Sign: -** If the most significant bit (MSB) of the result of the last operation is 1 (negative), S= 1 otherwise 0.
- Operation with Flag:
  - **STC** : Set Carry Flag (CY=1)
  - **CMC**: Complement Carry Flag (CY= $\overline{CY}$)

B. Arithmetic Operation

b) Arithmetic Operation Group

1. ADD R	2. ADD M	3. ADC R	4. ADC M
5. ADI data	6. ACI data	7. DAD R_P	8. SUB R
9. SUB M	10. SBB R	11. SBB M	12. SUI data
13. SBI data	14. DAA	15. INR R	16. INR M
17. DCR R	18. DCR M	19. INX R_P	20. DCX R_P

1. Addition with Accumulator

a) ADI 8-bit DATA
- Adds the 8 bit data with the contents of accumulator and stores result in accumulator.
- E.g. ADI 55H ; A ← A + 55H

b) ADD R/M
- Adds the contents of register/memory to the contents of the accumulator and stores the result in accumulator.
- E.g. ADD B ; A ← A+B
- ADD M ; A← A + M where M = [HL]

a) ACI 8-bit DATA
- Adds the 8 bit data with carry bit to the contents of accumulator and stores result in accumulator.
- E.g. ACI 55H ; A ← A + 55H + CY

b) ADC R/M
- Adds the contents of register/memory to the contents of the accumulator with carry bit and stores the result in accumulator.
- E.g. ADC B ; A ← A + B + CY

B. Arithmetic Operation

- Addition operation in 8085:

8085 performs addition with 8-bit binary numbers and stores the result in accumulator. If the sum is greater than 8-bits (FFH), it sets the carry flag.

E.g.
MVI A, 93H
MVI C, B7H
ADD C

```
  1 0 1 1 0 1 1 1        B7
+ 1 0 0 1 0 0 1 1      + 93
-----------------      ----
1 0 1 0 0 1 0 1 0      1 4A
  ^                    ^
  CY                   CY
```

B. Arithmetic Operation

2. Subtraction from Accumulator

a) SUI 8-bit DATA
- Subtracts the 8 bit data from the contents of accumulator and stores result in accumulator.
- E.g. SUI 77H ; A ← A - 77H

b) SUB R/M
- Subtracts the contents of specified register / memory from the contents of accumulator and stores the result in accumulator
- E.g. SUB D ; A ← A - D
- SUB M ; A← A - M where M = [HL]

a) SBI 8-bit DATA
- Subtracts the 8-bit immediate data and borrow (CY) from the content of the accumulator and stores the result in accumulator.
  E.g. SBI 77H ; A ← A - 77H - CY

d) SBB R/M
- Subtracts the contents of register or memory and borrow (CY) from the contents of accumulator and stores the result in accumulator.
  E.g. SBB D ; A ← A - D - CY

B. Arithmetic Operation

- Subtraction operation in 8085:

8085 performs subtraction operation by using 2's complement and the steps used are:

1) Converts the subtrahend (the number to be subtracted) into its 1's complement.
2) Adds 1 to 1's complement to obtain 2's complement of the subtrahend.
3) Adds 2's complement to the minuend (the contents of the accumulator).
4) Complements the carry flag.

B. Arithmetic Operation

A-B = A + (-B) = A + (2's complement of B)

- Subtraction operation in 8085:

E.g.
MVI A, 97H
MVI B, 65H
SUB B

65H: 0 1 1 0 0 1 0 1
1's complement of 65H : 1 0 0 1 1 0 1 0
+1
2's Complement of 65H: 1 0 0 1 1 0 1 1
97H: + 1 0 0 1 0 1 1 1
1 0 0 1 1 0 0 1 0
32
CY

Complement carry =0 ∴ **A=32 CY=0**

B. Arithmetic Operation

- Subtraction operation in 8085:

A-B = A + (-B) = A + (2's complement of B)

B=97H, A=65H

97H: 1 0 0 1 0 1 1 1

```
MVI A,65h
MVI B, 97H
SUB B
```

1's complement of 97H : 0 1 1 0 1 0 0 0

+1

2's Complement of 97H: 0 1 1 0 1 0 0 1

65H: + 0 1 1 0 0 1 0 1

0 1 1 0 0 1 1 1 0

(Result in 2's complement form)

CY

CY=1, A= CE: 1 1 0 0 1 1 1 1 0

1's complement: 0 0 1 1 0 0 0 0 1

2's complement: 0 0 1 1 0 0 0 1 0

32

B. Arithmetic Operation

3. 16 bit Addition

- DAD R_p
  - Double Addition
  - 1 byte instruction
  - Adds specified register pair with HL pair and store the 16 bit result in HL pair.
  - E.g. DAD B ; HL ← HL + BC

B. Arithmetic Operation

4. Increment and Decrement

a) INR R/M
   - Increase the contents of R(register) or M(memory) by 1
   - E.g. INR A ;A ← A +1
   - INR M ;[HL] ← [HL] + 1

b) DCR R/M
   - Decrease the contents of R(register) or M(memory) by 1
   - E.g. DCR A ; A← A-1
   - DCR C ; C ← C-1

c) INX R_p
   - Increase the register pair by 1
   - E.g. INX B ; BC ← BC +1

d) DCX R_p
   - decrease the register pair by 1
   - E.g. DCX D ;DE ← DE-1

B. Arithmetic Operation

5. DAA

- **Decimal adjustment accumulator**
- Used only after addition
- 1 byte instruction
- The content of accumulator is changed from binary to two 4-bit BCD digits.
- E.g.

```
MVI A, 25H ; A ← 25H
MVI B, 16H  ; B ← 16H
ADD B       ; A ← A + B = 25H+16H = 3BH
DAA         ; A← 41H
```

C. Logical Operation

- A microprocessor is basically a programmable logic chip. It can perform all the logic functions of the hardwired logic through its instruction set.
- The 8085 instruction set includes such logic functions as AND, OR, XOR and NOT (Complement)
- The following features hold true for all logic instructions:
  - The instructions implicitly assume that the accumulator is one of the operands.
  - All instructions reset (clear) carry flag except for complement where flag remain unchanged.
  - They modify Z, P & S flags according to the data conditions of the result.
  - Place the result in the accumulator.
  - They do not affect the contents of the operand register

C. Logical Operation

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

X ANDing by 0 -> value reset (0)
X ANDing by 1 -> Value remains same (X)

1. Logic function operation
   a. **ANA R/M**
      - Logically AND the contents of register/memory with the contents of accumulator.
      - 1 byte instruction.
      - CY flag is reset and AC is set.

   b. **ANI 8-bit DATA**
      - Logically AND 8 bit immediate data with the contents of accumulator.
      - 2 byte instruction.
      - CY flag is reset and AC is set. Others as per result

C. Logical Operation

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

X ORing by 0 -> Value remains same (X)
X ORing by 1 -> Value Set (1)

1. Logic function operation

c. ORA R/M

- Logically OR the contents of register/memory with the contents of accumulator.
- 1 byte instruction.
- CY and AC is reset and other as per result.

d. ORI 8 bit data

- Logically OR 8 bit immediate data with the contents of the accumulator.
- 2 byte instruction.
- CY and AC is reset and other as per result.

C. Logical Operation

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

X XORing by 0 -> Value remains same
(X) X XORing by 1 -> Value Complement ($\bar{X}$)

1. Logic function operation

e. XRA R/M

- Logically exclusive OR the contents of register memory with the contents of accumulator.
- 1 byte instruction.
- CY and AC is reset and other as per result.

f. XRI 8 bit data

- Logically Exclusive OR 8 bit data immediate with the content of accumulator.
- 2 byte instruction.
- CY and AC is reset and other as per result.

C. Logical Operation

1. Logic function operation

g. CMA

- Complement accumulator
- 1 byte instruction.
- Complements the contents of the accumulator.
- No flags are affected.

C. Logical Operation

2. Logically Compare instructions

a. **CMP R/M** (1 byte instruction)

b. **CPI 8 bit data** (2 byte instruction)

- Compare the contents of register/ memory and 8 bit data with the contents of accumulator.
- Status is shown by flags & all flags are modified.

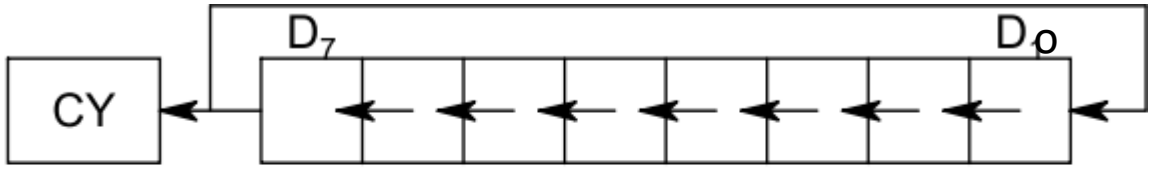
Case	CY	Z	
[A]<[R/M] or 8 bit	1	0	A-R<0
[A]=[R/M] or 8 bit	0	1	A-R=0
[A]>[R/M] or 8 bit	0	0	A-R>0

C. Logical Operation

3. Logical Rotate instructions

a. RLC

- Rotate accumulator left
- Each bit is shifted to the adjacent left position. Bit D7 becomes D0.
- The carry flag is modified according to D7.



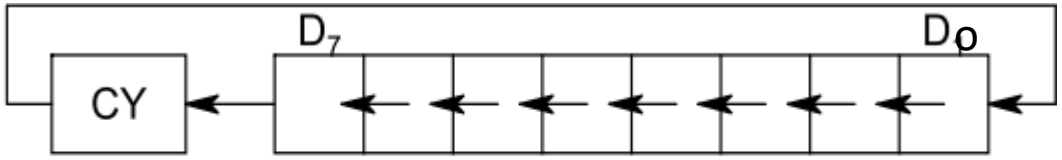
CY= D_7, D_7= D_6, D_6=D_5,……,D_1=D_0, D_0=D_7

C. Logical Operation

3. Logical Rotate instructions

b. RAL

- Rotate accumulator left through carry
- Each bit is shifted to the adjacent left position. Bit D7 becomes the carry bit and the carry bit is shifted into D0.
- The carry flag is modified according to D7.



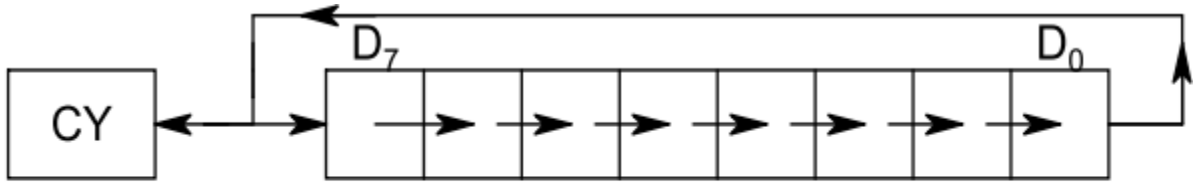
$CY = D_7, D_7 = D_6, D_6 = D_5, \ldots\ldots, D_1 = D_0, D_0 = CY$

C. Logical Operation

3. Logical Rotate instructions

c. RRC

- Rotate accumulator right
- Each bit is shifted right to the adjacent position. Bit D0 becomes D7
- The carry flag is modified according to D0.



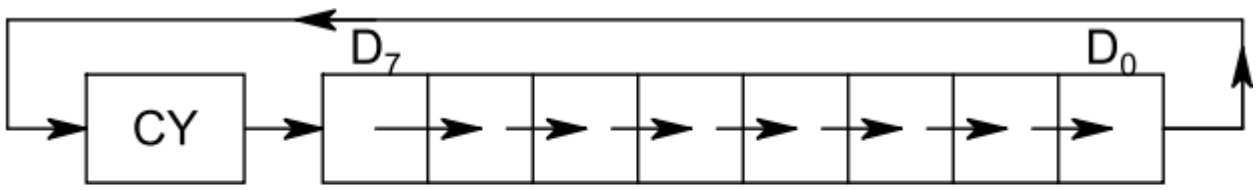
$CY = D_0$, $D_7 = D_0$,……,$D_0 = D_1$

C. Logical Operation

3. Logical Rotate instructions

d. RAR

- Rotate accumulator right through carry
- Each bit is shifted right to the adjacent position.
- Bit D0 becomes the carry bit and the carry bit is shifted into D7.



$CY = D_0$, $D_0 = D_1$, ……..$D_7 = CY$

D. Branching Instructions

- Microprocessor is a sequential machine; it executes machine codes from one memory location to the next.
- The branching instructions instruct the microprocessor to go to a different memory location and the microprocessor continues executing machine codes from that new location
- The branching instructions are the most powerful instructions because they allow the microprocessor to change the sequence of a program, either unconditionally or under certain test conditions.
- The branching instruction code categorized in following three groups:
  - Jump instructions
  - Call and return instruction
  - Restart instruction

- CC addr - Call, if carry flag is set.
- CNC addr - Call, if carry flag is not set (reset).
- CZ addr - Call, if zero flag is set.
- CNZ addr - Call, if zero flag is not set (reset).
- CP addr - Call, if positive i.e. sign flag is reset.
- CM addr - Call, if minus i.e. sign flag is set.
- CPE addr - Call, if parity even i.e. parity flag is set.
- CPO addr - Call, if parity odd i.e. parity flag is reset.

D. Branching Instructions

1. Jump Instruction

- Jump instructions specify the memory location explicitly.
- They are 3 byte instructions, one byte for the operation code followed by a 16 bit (2 byte) memory address.
- Jump instructions can be categorized into unconditional and conditional jump.
    a. Unconditional Jump:
        - To enable the programmer to set up continuous loops without depending only type of conditions
        - **JMP 16-bit ADDRESS**
        - loads the program counter by 16 bit address and jumps to specified memory location.
        - The jump location can also be specified using a label or name.
        - **JMP Label**

D. Branching Instructions

1. Jump Instruction
   a. Unconditional Jump:

e.g.

Address	Mnemonics	Hexcode
8000	MVI B,FF	06
8001		FF
8002	DCR B	05
8003	JMP 8002	C3
8004		02
8005		80
8006	HLT	76

Address	Mnemonics	Hexcode
8000	MVI B,FF	06
8001		FF
8002	L1: DCR B	05
8003	JMP L1	C3
8004		02
8005		80
8006	HLT	76

D. Branching Instructions

1. Jump Instruction

b. Conditional Jump:

- Conditional jump instructions allow the microprocessor to make decisions based on certain conditions indicated by the flags.
- After logic and arithmetic operations, flags are set or reset to reflect the condition of data.
- These instructions check the flag conditions and make decisions to change or not to change the sequence of program.
- The four flags namely carry, zero, sign and parity used by the jump instruction.

D. Branching Instructions

1. Jump Instruction

b. Conditional Jump:

Mnemonics	Description
JC 16-bit ADDRESS/Label	Jump on Carry (if **CY=1**, Jump)
JNC 16-bit ADDRESS/Label	Jump on No Carry (If **CY=0**, Jump)
JZ 16-bit ADDRESS/Label	Jump on Zero (If **Z=1**, Jump)
JNZ 16-bit ADDRESS/Label	Jump on No Zero (If **Z=0**, Jump)
JP 16-bit ADDRESS/Label	Jump on Plus (if **S=0**, Jump)
JM 16-bit ADDRESS/Label	Jump on Minus (If **S=1**, Jump)
JPE 16-bit ADDRESS/Label	Jump on Parity Even (If **P=1**, Jump)
JPO 16-bit ADDRESS/Label	Jump on Parity Odd (If **P=0**, Jump)

D. Branching Instructions

1. Jump Instruction

   b. Conditional Jump:

Address	Mnemonics	Hexcode
8000	MVI B,FF	06
8001		FF
8002	L1: DCR B	05
8003	JNZ L1	C3
8004		02
8005		80
8006	HLT	76

for(b=255; b>=0; b--)

D. Branching Instructions

2. Call and Return Instructions

- Call and return instructions are associated with subroutine technique.
- A subroutine is a group of instructions that perform a subtask. A subroutine is written as a separate unit apart from the main program and the microprocessor transfers the program execution sequence from main program to subroutine whenever it is called to perform a task.
- After the completion of subroutine task microprocessor returns to main program.
- The subroutine technique eliminates the need to write a subtask repeatedly, thus it uses memory efficiently.
- Before implementing subroutine, the stack must be defined; the stack is used to store the memory address of the instruction in the main program that follows the subroutines call.

D. Branching Instructions

2. Call and Return Instructions

To implement subroutine there are two instructions CALL and RET.

a. Unconditional Call and Return

- **CALL** 16-bit ADDRESS/Label
  - Call subroutine unconditionally
  - Saves the contents of program counter on the stack pointer. Loads the PC by jump address (16 bit memory) and executes the subroutine.
  - 3 byte Instruction

- **RET**
  - Returns from the subroutine unconditionally
  - Inserts the contents of stack pointer to program counter.
  - One byte instruction

D. Branching Instructions

2. Call and Return Instructions

a. Unconditional Call and Return

E.g.

Address	Mnemonics	Hexcode
8000	MVI A, 05	3E
8001		05
8002	L1: OUT 40H	D3
8003		40
8004	CALL DELAY	CD
8005		00
8006		90
8007	DCR A	3D
8008	JNZ L1	C3
8009		02
800A		80
800B	HLT	76
9000	DELAY: MVI B,FFH	06
9001		FF
9002	L2: DCR B	05
9003	JNZ L2	C3
9004		02
9005		90
9006	RET	76

D. Branching Instructions

2. Call and Return Instructions

b. Conditional Call and Return

Mnemonics	Description
CC 16-bit ADDRESS/Label	Call on Carry (if **CY=1**, Call)
CNC 16-bit ADDRESS/Label	Call on No Carry (If **CY=0**, Call)
CZ 16-bit ADDRESS/Label	Call on Zero (If **Z=1**, Call)
CNZ 16-bit ADDRESS/Label	Call on No Zero (If **Z=0**, Call)
CP 16-bit ADDRESS/Label	Call on Plus (if **S=0**, Call)
CM 16-bit ADDRESS/Label	Call on Minus (If **S=1**, Call)
CPE 16-bit ADDRESS/Label	Call on Parity Even (If **P=1**, Call)
CPO 16-bit ADDRESS/Label	Call on Parity Odd (If **P=0**, Call)

D. Branching Instructions

2. Call and Return Instructions

b. Conditional Call and Return

Mnemonics	Description
RC	Return on Carry (if **CY=1**, Return)
RNC	Return on No Carry (If **CY=0**, Return)
RZ	Return on Zero (If **Z=1**, Return)
RNZ	Return on No Zero (If **Z=0**, Return)
RP	Return on Plus (if **S=0**, Return)
RM	Return on Minus (If **S=1**, Return)
RPE	Return on Parity Even (If **P=1**, Return)
RPO	Return on Parity Odd (If **P=0**, Return)

D. Branching Instructions

3. Restart Instructions

- 8085 instruction set includes 8 restart instructions (RST).
- These are 1 byte instructions and transfer the program execution to a specific location.
- When RST instruction is executed, the 8085 stores the contents of PC on SP and transfers the program to the restart location.
- Actually these restart instructions are inserted through additional hardware.
- These instructions are part of interrupt process.

D. Branching Instructions

3. Restart Instructions

Restart instruction	Hex Code	Call location in hex
RST 0	C7	0000H
RST 1	CF	0008H
RST 2	D7	0010H
RST 3	DF	0018H
RST 4	E7	0020H
RST 5	EF	0028H
RST 6	F7	0030H
RST 7	FF	0038H

D. Miscellaneous Instructions:

1. Stack Operation
   - Stack is defined as a set of memory location in R/W memory, specified by a programmer in a main memory.
   - Stack memory locations are used to store binary information temporarily during the execution of a program
   - The beginning of the stack is defined in the program by using the instruction: **LXI SP, 16 bit address**
   - Once the stack location is defined, it loads 16 bit address in the stack pointer register.
   - Storing of data bytes for this operation takes place at the memory location that is one less than the address
   - The stack is initialized at the highest available memory location to prevent the program from being destroyed by the stack information

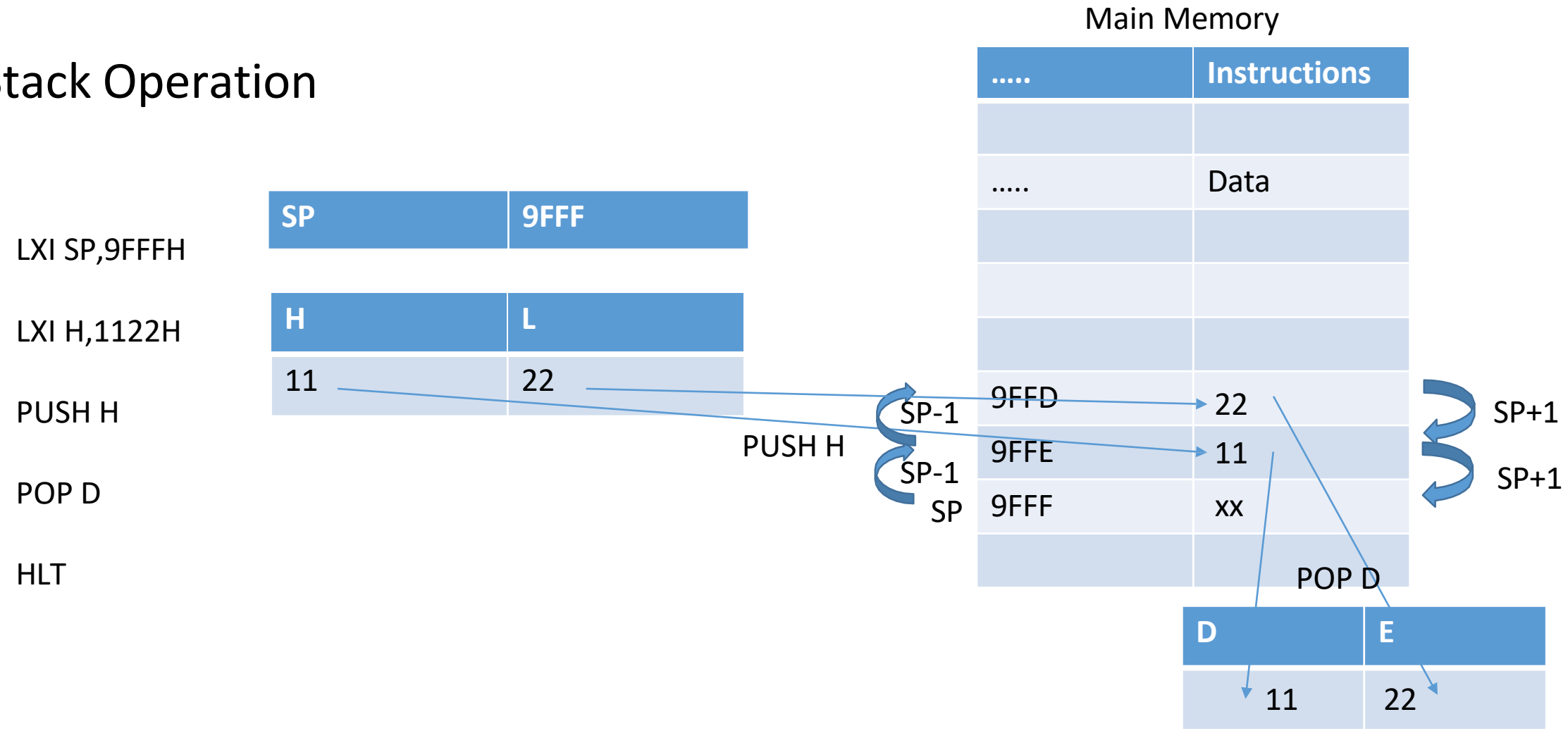
D. Miscellaneous Instructions:

1. Stack Operation
   a) **PUSH Rp/PSW**
      - Store register pair on stack
      - 1 byte instruction
      - Copies the contents of specified register pair or program status word (accumulator and flag) on the stack.
      - Stack pointer is decremented by 1 and content of high order register is copied. Then it is again decremented by 1 and content of low order register is copied.

   b) **POP Rp/PSW**
      - retrieve register pair from stack
      - 1 byte instruction.
      - Copies the contents of the top two memory locations of the stack into specified register pair or program status word.
      - A content of memory location indicated by SP is copied into low order register and SP is incremented by 1. Then the content of next memory location is copied into high order register and SP is incremented by 1.

D. Miscellaneous Instructions:

1. Stack Operation



D. Miscellaneous Instructions:

1. Stack Operation

d) **XTHL** – exchanges top of stack (TOS) with HL (HL←→SP)

e) SPHL – move HL to SP (SP ← HL)

f) PCHL – move HL to PC (PC ← HL)

D. Miscellaneous Instructions:

2. Instructions related to interrupt

- **DI** – disable interrupt
- **EI** – Enable interrupt
- **SIM** – set interrupt mask
- **RIM** – read interrupt mask

End of Chapter two !